

Important: Please ensure that you use the same e-mail ID that you used to create the GitHub account.

```
[pmu@t430 ~]$ git config --global user.name "pmu"
[pmu@t430 ~]$ git config --global user.email "pmu.rwx@gmail.com"
```

Check the Git man page to see what other options can be set. Once done, you can check the options with the following command:

```
[pmu@t430 ~]$ git config --list
```

Initialising Git

We will now create a directory with the same name as that of the repository.

```
[pmu@t430 ~]$ mkdir mydotfiles
```

Let's get into the directory and initialise Git by running the `git init` command. Once you do that, there should be a `.git` directory created in there with a few files and directories under it. In Git terminology, the `mydotfiles` directory has now become a working tree. Now, the `mydotfiles` directory and everything under it can be uploaded to Git.

Creating files for some Git action

We will create the `Readme.txt` file now and add some content to it. I created a `Readme.txt` file in Vim, saved it and then ran the `more` command to show the output in the terminal.

```
[pmu@t430 mydotfiles]$ vim Readme.txt
[pmu@t430 mydotfiles]$ more Readme.txt
```

This is a `Readme` file, my first file that I'll try to upload on GitHub.

```
[pmu@t430 mydotfiles]$
```

You can add anything you want to the file.

Now let's run the `git status` command. This is a very helpful command that lets us know exactly what stage we are at.

Now let's look at the concept that's unique to Git. The moment we add a file, Git creates a hash checksum and refers to the file using that checksum. In other words, we call the file `Readme.txt` but Git refers to it by its checksum. If we were to look for files in a directory, we'd run the `ls` command. With Git, we run the `Git ls-files --stage` command.

```
[pmu@t430 mydotfiles]$ git ls-files --stage
100644 b70f72952f495b2aae83f2ff1a50b5ee8d001edb 0 Readme.txt
[pmu@t430 mydotfiles]$
```

Look at the long string -

`b70f72952f495b2aae83f2ff1a50b5ee8d001edb`. That's how Git refers to what we call the `Readme.txt` file. So what does this file have? Well, we could check it as follows:

```
[pmu@t430 mydotfiles]$ git show b70f
```

This is a `Readme` file, my first file that I'll try to upload on GitHub.

```
[pmu@t430 mydotfiles]$
```

That's the Git equivalent of more `Readme.txt` or `cat Readme.txt`. That's exactly the same content we have in our `Readme.txt` file. Note that we can just supply the first four characters of the hash. So this means that Git really doesn't track or manage a file using the filename we give to it. It just 'cares about' the hashed checksum. In other words, if two or more files were to have the exact same content, then the hash checksum generated for those files will be exactly the same.

The following example will make it even clearer.

Let's copy the `Readme.txt` file with some other name without changing any content inside the actual file.

```
[pmu@t430 mydotfiles]$ cp Readme.txt Oncemore.txt
```

```
[pmu@t430 mydotfiles]$ ls -l
total 8
-rw-rw-r-- 1 pmu pmu 72 Sep 22 21:03 Oncemore.txt
-rw-rw-r-- 1 pmu pmu 72 Sep 22 20:53 Readme.txt
```

```
[pmu@t430 mydotfiles]$
```

Let's run the `Git status` command and see how it responds.

```
[pmu@t430 mydotfiles]$ git status
# On branch master
# Initial commit
# Changes to be committed:
#   (use "git rm --cached <file>..." to unstage)
#   new file:   Readme.txt
# Untracked files:
#   (use "git add <file>..." to include in what will be committed)
#   Oncemore.txt
[pmu@t430 mydotfiles]$
```

It shows the newly created file. Note that it states `Readme.txt` as a new file, because we just added it with the `git add Readme.txt` command. We also have a copy of that file with the name `Oncemore.txt`. Since we have still not added it, it shows up as an untracked file.

Will Git show some different output with the `Git ls-files --stage` command this time?